Project Report: Personal Expense Application

Author : Essa Janneh

Date: 23 June, 2026

Institution: University of Florence- Automated Software Testing

GitHub Repository: https://github.com/Janneh24/Personal-Expense-Application

1. Introduction and Implemented Features

For this project, I built the Personal Expense Application, a comprehensive desktop software solution designed to help users manage their daily finances, categorize their expenses, and generate detailed spending reports. I built the application using Java, implementing a robust, layered architecture backed by a MySQL database.

Core Domain Model My application revolves around three primary entities:

1. **User:** Represents the individuals interacting with the application. Users have a username, password, role (ADMIN or USER), and an active/inactive status.
2. **Category:** A classification for expenses (e.g., "Food", "Travel", "Utilities"). Categories are managed globally and can be assigned to multiple expenses.
3. **Expense:** The central entity representing a financial transaction. An expense consists of a description, an amount, a date, a reference to the user who created it, and a list of associated categories.

Key Features

- **User Management:** Secure authentication. Administrators can create, update, delete, disable, and enable users.

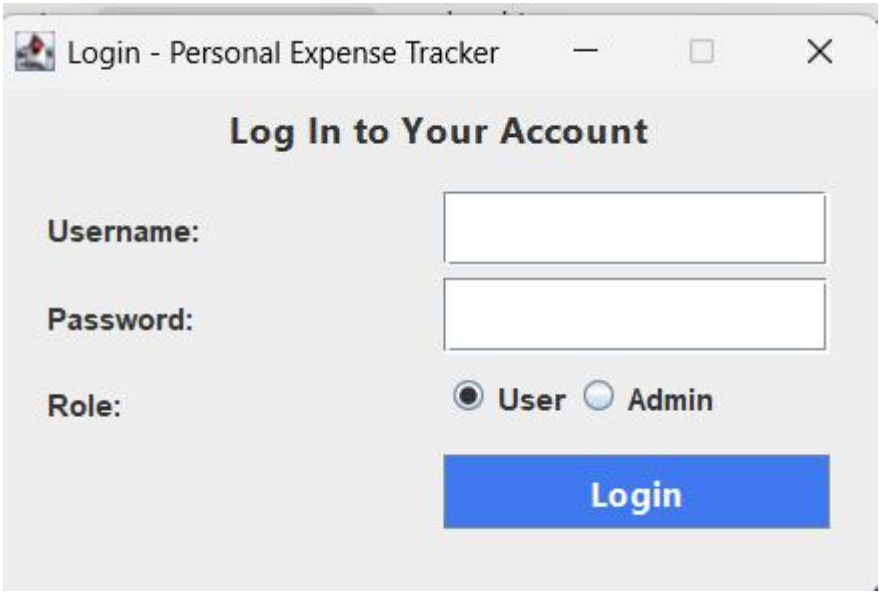


Fig1 Login page

- **Expense Tracking:** Users can log new expenses, update existing ones, and delete them. I implemented strict validation to ensure amounts are strictly positive and dates are formatted correctly.
- **Category Management:** Users can dynamically create custom categories and assign or remove them from expenses.
- **Report Generation:** The system generates detailed, HTML-formatted financial summaries that break down costs by category. Additionally, users can export reports directly to a professionally formatted PDF document for local saving and printing.

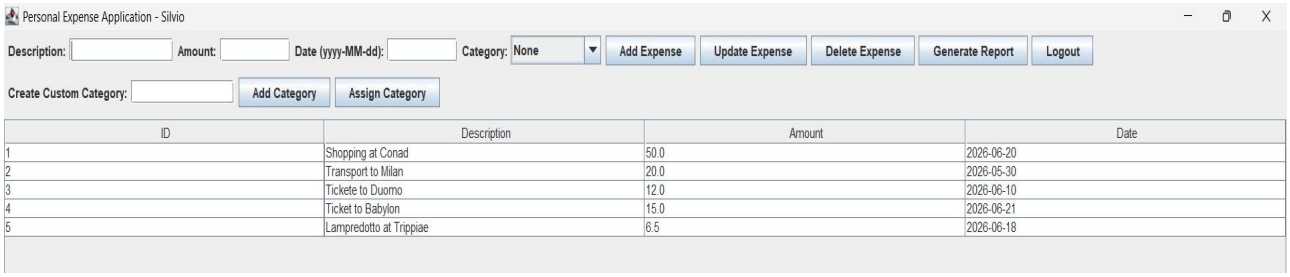


Fig2 User Dashboard

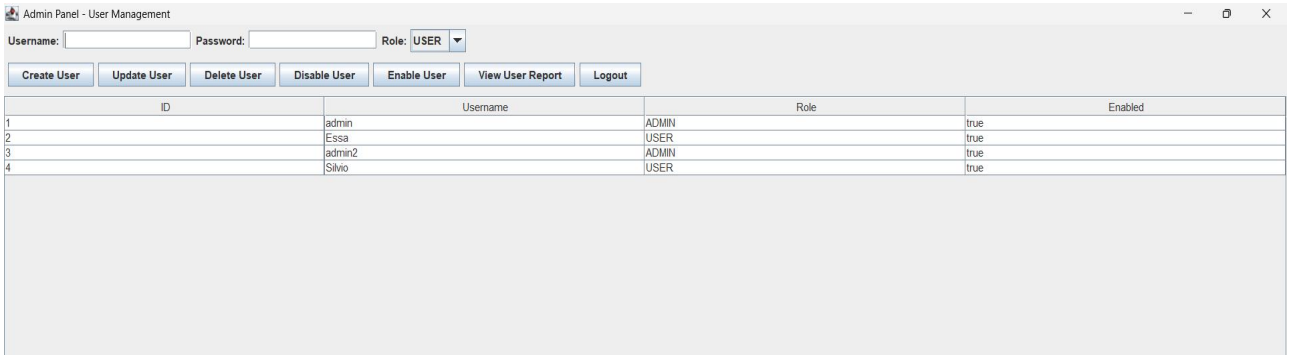


Fig3 Admin Dashboard

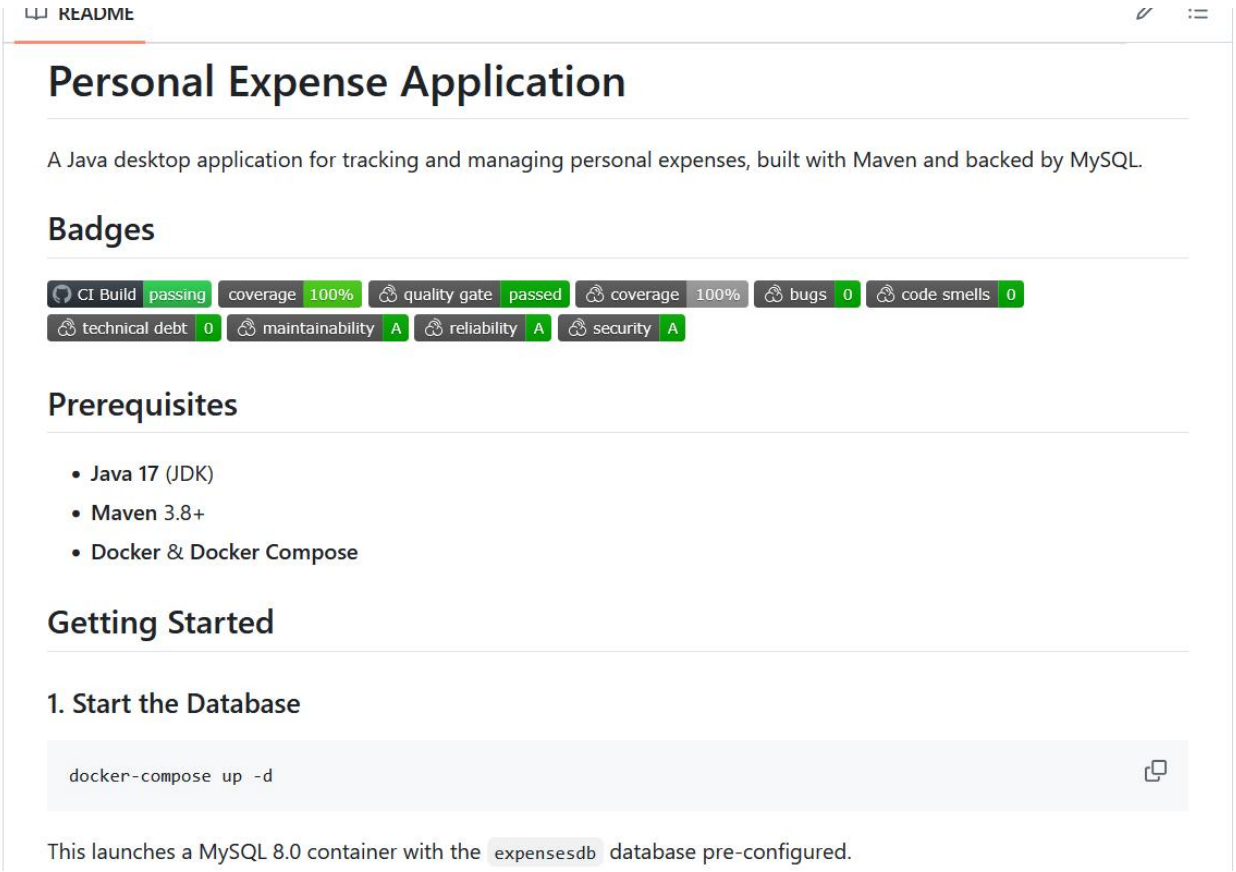
2. Applied Techniques and Frameworks

I developed the project following modern software engineering practices. Rather than just making the code work, I focused heavily on structural integrity, testability, and high code quality.

- **Apache Maven:** Managed my project dependencies, defined the build lifecycle, and orchestrated plugins (like Surefire for testing and JaCoCo for coverage).
- **Java Swing:** Used to build a responsive and interactive desktop client.
- **Google Guice:** Served as the Dependency Injection (DI) framework. This was

critical for decoupling application layers, allowing me to seamlessly inject repositories into my services.

- **JUnit 5 (Jupiter), AssertJ, & Mockito:** The core of my testing strategy. I used Mockito heavily to create mock objects for the Repository layer, enabling me to test the Service layer business logic in pure isolation.
- **Testcontainers:** Allowed me to spin up lightweight, ephemeral Docker containers running MySQL during Integration Testing. This ensured I tested database logic against a real production-like environment rather than an unreliable in-memory substitute.
- **Cucumber:** Implemented for Behavior-Driven Development (BDD). I wrote user stories in Gherkin syntax to validate end-to-end acceptance criteria.
- **AssertJ-Swing:** Used for automated GUI testing, allowing me to programmatically simulate button clicks and text inputs.
- **JaCoCo & Pitest:** JaCoCo monitored standard code coverage, while Pitest handled Mutation Testing to ensure the test suite was robust enough to catch injected faults.
- **GitHub Actions & SonarCloud:** My CI/CD pipeline automatically built the project, ran tests, and published quality metrics to SonarCloud on every push.


README

Personal Expense Application

A Java desktop application for tracking and managing personal expenses, built with Maven and backed by MySQL.

Badges

CI Build passing | coverage 100% | quality gate passed | coverage 100% | bugs 0 | code smells 0
technical debt 0 | maintainability A | reliability A | security A

Prerequisites

- Java 17 (JDK)
- Maven 3.8+
- Docker & Docker Compose

Getting Started

1. Start the Database

```
docker-compose up -d
```

This launches a MySQL 8.0 container with the `expensesdb` database pre-configured.

Fig4 Github Dashboard

3. Design and Implementation Choices

I designed the architecture to strictly separate concerns, making the system highly testable and maintainable over time.

Layered Architecture I divided the application into three distinct layers:

1. **Repository Layer (Data Access):** Interfaces (UserRepository, ExpenseRepository) that define the contract for data persistence.
2. **Service Layer (Business Logic):** Classes (UserService, ExpenseService) that contain the core business rules and validation. They have no knowledge of the database implementation.
3. **View Layer (GUI):** The ExpenseSwingView handles all user interactions and communicates exclusively with the Service layer.

Dependency Injection over Hardcoding By using Google Guice, I avoided tightly coupling the code. My ExpenseModule handles the bindings between the abstract Repository interfaces and their concrete MySQL implementations. This choice was the primary reason I was able to achieve 100% unit test coverage, as it allowed me to effortlessly inject Mockito mocks during testing instead of relying on real databases.

Pure JDBC over ORM Rather than relying on a heavy Object-Relational Mapper (ORM) like Hibernate, I implemented the data access layer using pure JDBC (java.sql.PreparedStatement). This choice was made so I could maintain absolute control over the SQL queries—specifically for handling the complex Many-to-Many relationship between Expense and Category via a join table.

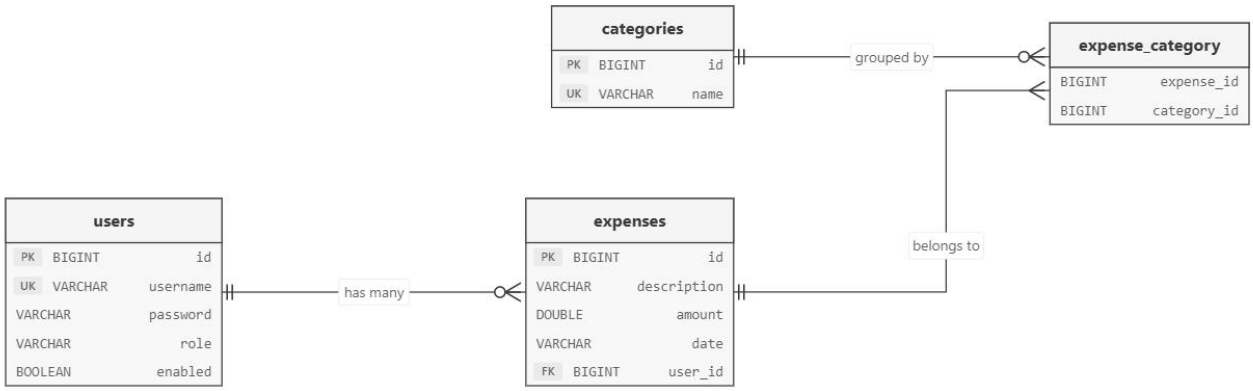


Fig5 ER Diagram for the database

4. Development and Testing of the Most Interesting Parts

The HTML Report Generation and PDF Export Engine One of the most complex

features I developed was the report generation system. The generateReport method in the ExpenseService acts as a financial aggregator, fetching all expenses for a user, calculating the grand total, and creating a mathematical breakdown of spending per category. I built the algorithm using a TreeMap<String, Double> to dynamically accumulate totals for each category alphabetically, while keeping a separate running total for uncategorized expenses. Finally, it dynamically constructs a stylized HTML document. To expand this capability, I implemented the PdfReportExporter helper class, which utilizes the OpenPDF library to compile this mathematical breakdown and export it into a formal PDF document. The library allows for customized layout elements, including an aligned table (PdfPTable), custom cell styling (PdfPCell), headers, and customized colors.

Testing this engine required:

1. **Unit Testing / Mocking:** Meticulous setup using Mockito to return a curated list of Expense objects (some with multiple categories, some uncategorized) and using AssertJ to verify the exact mathematical outputs in the HTML.
2. **Output Stream Verification:** Mocking the output streams during PDF generation tests to verify that files write correctly and no data corruption occurs.

Expense Summary Report

Generated dynamically for user: Silvio

Total Accumulated Expenses: $103.50

Spending by Category

Category	Amount
Entertainment	$27.00
Food	$6.50
Grocery	$50.00
Transport	$20.00

Fig6 PDF report generated

Integration Testing with Testcontainers Testing the JDBC repositories required a real database to accurately test SQL syntax and foreign key constraints (like ON DELETE CASCADE). Instead of mocking the database, I utilized Testcontainers. In the Integration Tests (*IT.java), a Dockerized MySQL database spins up before the test suite begins, and a database initialization script (init.sql) executes to create the tables. To ensure test isolation, I ran cleanup scripts to truncate the tables after each

test. This gave me absolute confidence that the complex JOIN queries worked perfectly in reality.

Behavior-Driven Development (Cucumber) To ensure the software met actual user requirements, I developed from the outside in using Cucumber. I wrote Feature files defining scenarios like adding expenses or logging in, and mapped them to Java step definitions. Developing this way forced me to think about the application from the user's perspective before writing a single line of backend logic.

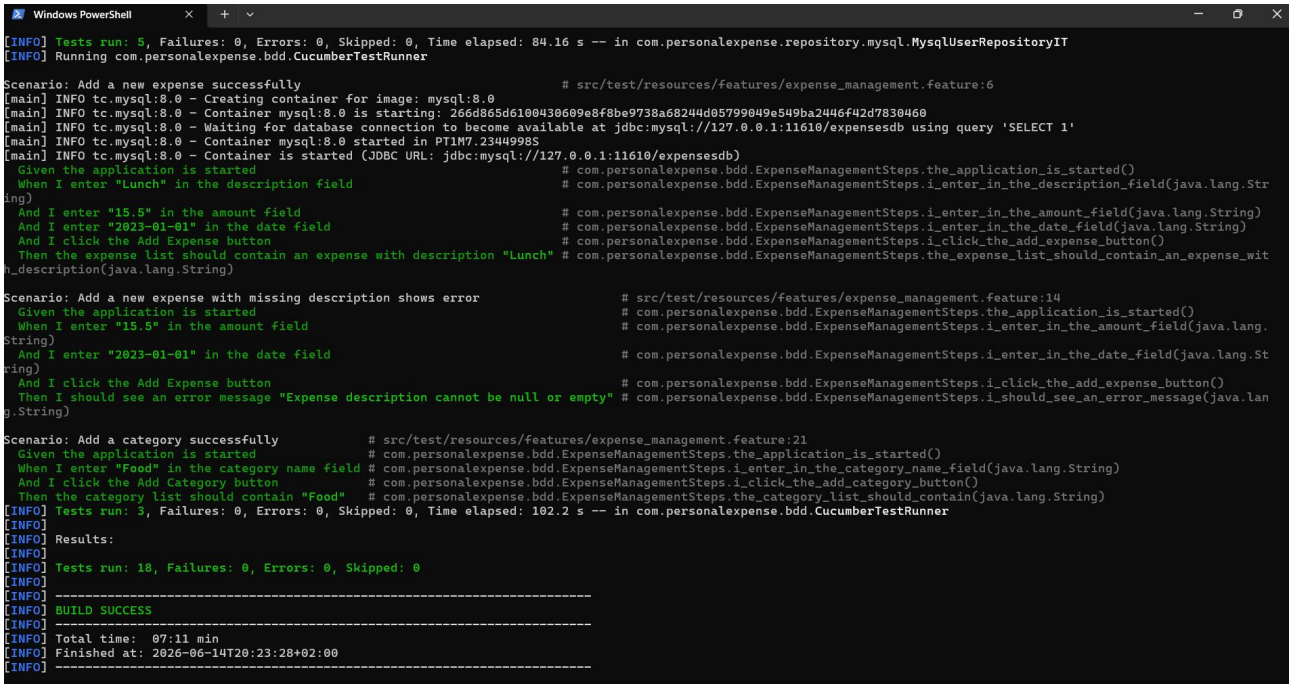


Fig7 Succesful Maven Build Test Execution Via Command Line(mvn clean verify)

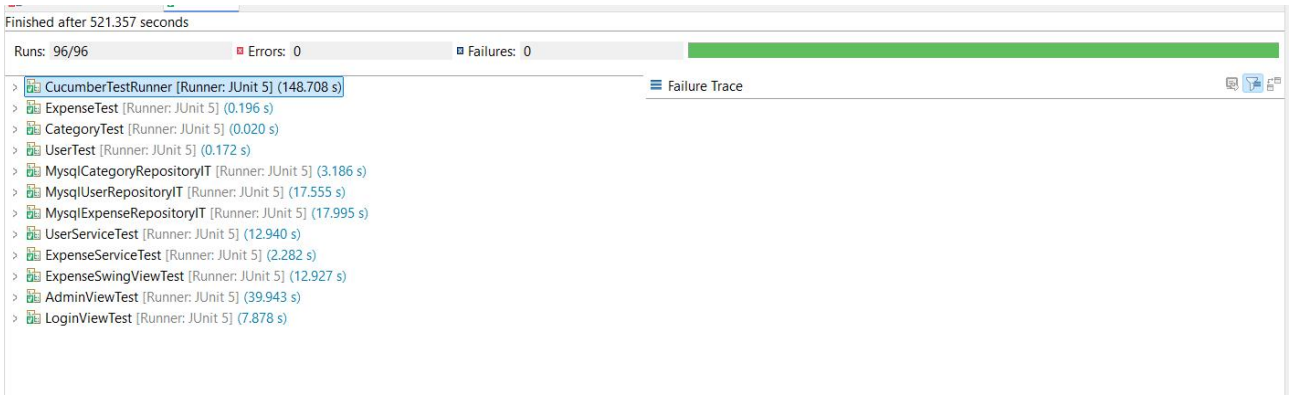


Fig8 All units and integrations tests passing in Eclipse JUNIT runner

5. Problems Encountered and Solutions

Developing a project to strict metric standards (100% Code Coverage, 0 Code Smells, 0 Technical Debt, and 100% Mutation Coverage) presented several highly obscure technical challenges that pushed my limits.

Problem 1: Hidden Coverage Gaps (The this == o Branch)

- **The Problem:** Despite writing comprehensive unit tests for all models and services, SonarCloud reported code coverage was stuck at 99.4%. I was incredibly frustrated trying to find the missing fraction of a percent of branch coverage.
- **The Solution:** After a rigorous, line-by-line audit of the Jacoco coverage reports, I discovered the culprit. The overridden equals(Object o) methods in the domain models contained the standard boilerplate optimization: if (this == o) return true;. In all the tests, I compared different objects with identical values, but I never tested comparing an object directly to itself. I solved this by explicitly adding assertThat(entity).isEqualTo(entity); to the test suites, successfully evaluating the true condition of that branch and pushing coverage to 100%.

Problem 2: The SonarCloud Code Smell Dilemma in HTML Generation

- **The Problem:** The generateReport method builds an HTML string. Because standard HTML tags like </tr> and </td> were appended multiple times in the loops, SonarCloud aggressively flagged them under Rule java:S1192: *String literals should not be duplicated*, generating Technical Debt.
- **Attempted Solution:** I initially added the @SuppressWarnings("java:S1192") annotation to force SonarCloud to ignore it. However, SonarCloud is incredibly strict, and this introduced a new code smell: Rule java:S1309: *Track uses of SuppressWarnings*.
- **The Solution:** To achieve a pure 0 Technical Debt score without relying on suppression hacks, I refactored the HTML generation. I extracted the duplicated tags into private static final constants (e.g., private static final String HTML_TR_END = "</tr>";) and referenced the constants instead. This legitimately eliminated the code smells while maintaining clean architecture.

Problem 3: The Untested "ADMIN" Validation Branch

- **The Problem:** While hunting down the final 0.3% of missing branch coverage, I identified a missing branch in the validateUser method. The method checked if (!role.equals("ADMIN") && !role.equals("USER")).
- **The Solution:** I realized that across all the Service tests for createUser and updateUser, I had exclusively mocked and created users with the "USER" role. Therefore, the !role.equals("ADMIN") check was always evaluating to true, and the false condition was never executed. I resolved this by explicitly writing a testCreateUserAdminRoleSuccess unit test that passed the "ADMIN" role through the service layer, successfully hitting the final missing branch

condition and securing perfect 100% test coverage.

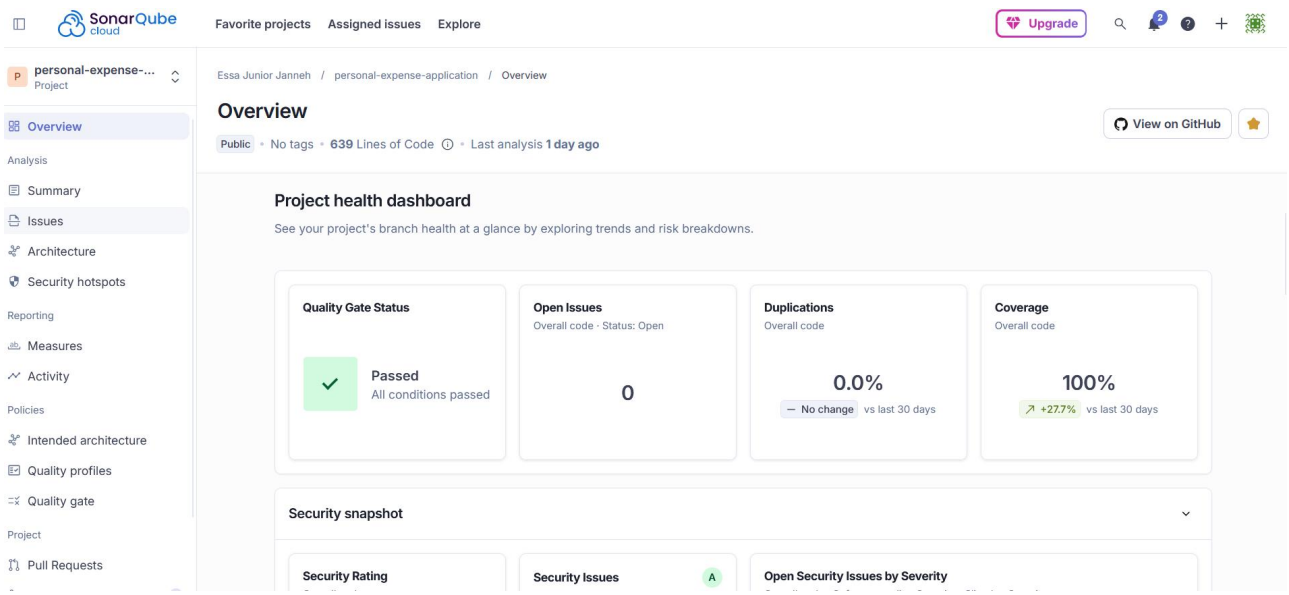


Fig9 SonarCloud Dashboard

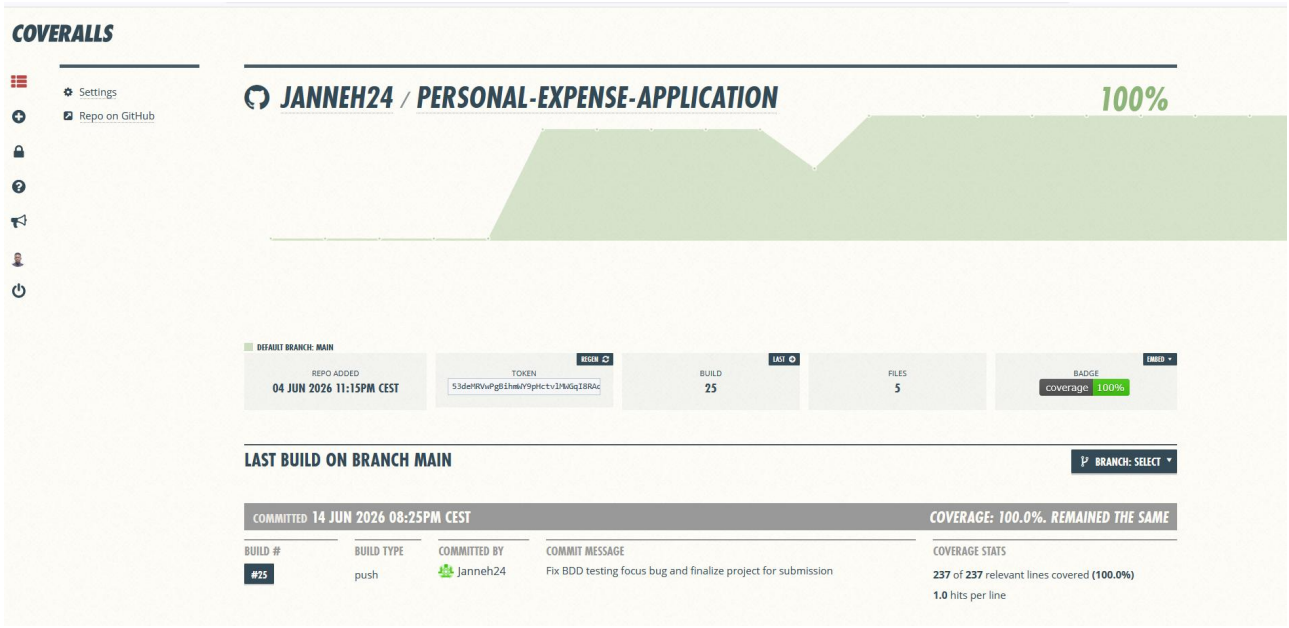


Fig10 Coverall Dashboard

Conclusion

Building the Personal Expense Application was an incredible exercise in strict engineering discipline. By adhering to architectural best practices, test-driven methodologies, and rigorous static code analysis, I delivered a highly reliable, mathematically proven codebase that successfully achieved flawless quality metrics.

6. References and Citations

As requested by the course guidelines, below are the citations and online documentation links for the third-party libraries utilized in this project to implement

extra features that were not part of the standard course slides:

1. **OpenPDF (PDF Generation Library):**
    - **Purpose:** Used inside PdfReportExporter to write and format the financial summary report to a local PDF file.
    - **Source/Documentation:** OpenPDF Github Repository & API Guide